A 1.5-μJ/Task Path-Planning Processor for 2-D/3-D Autonomous Navigation of Microrobots

Chieh Chung, *Graduate Student Member, IEEE*, and Chia-Hsiang Yang, *Senior Member, IEEE*

***Abstract*— Autonomous microrobots have been utilized in a wide range of applications. Energy-efficient, real-time path planning for navigation is essential. This work presents a path-planning processor for 2-D/3-D autonomous navigation. Energy and latency are minimized through algorithm-architecture optimization. The processor utilizes the rapidly exploring random tree (RRT) algorithm to ensure efficient planning on maps that have higher dimensions and a higher resolution. Dual-tree planning, branch extension, and parallel expansion are adopted in order to reduce both computational complexity and memory requirements. A prune-and-reuse strategy is also adopted so as to quickly respond to dynamic scenarios. An array of processing engines (PEs) is deployed in order to enable parallel expansion. The number of PEs is minimized through latency analysis. Low-complexity implementation for the PE is proposed while maintaining a high performance. Fabricated in a 40-nm CMOS technology, the chip integrates 2M logic gates in an area of 3.65 mm^2. The processor supports path-planning tasks for both 2-D and 3-D maps, with latencies of less than 1 and 10 ms, respectively. For a 2-D map that has 100 × 100 grids, the proposed processor dissipates 1.5 μ/task at a clock frequency of 200 MHz from a 0.9-V supply. Compared with the state-of-the-art designs, the proposed path-planning processor achieves a 1467× shorter processing latency based on an energy dissipation that is 2133× lower, despite the capability for larger maps.**

***Index Terms*— Autonomous navigation, digital integrated circuits, microrobot, path planning, rapidly exploring random tree (RRT).**

I. Introduction

RECENT advances in the design of autonomous microrobots, as well as the associated technology have changed the way humans live and work. Robots are now able to assist humans or even substitute them in many applications. For example, industrial robots move on their own and perform package delivery [1], [2]. Drones can fly autonomously to perform intelligent surveillance [3], [4]. Autonomous terrain exploration can also be performed [5], [6]. For autonomous navigation, robots need to perceive the environment, make decisions based on the information collected, and autonomously take action [7]. In the perception phase, the sensed data are analyzed to create a map. In the cognition phase, the actions of the robot need to be determined based on its objectives. The tasks include identifying a path, from a given start and endpoint, without colliding with any obstacles. In the action phase, motion control commands are executed so as to guide the robots along the safest path.

To accelerate path planning, graphics processing units (GPUs) have been proposed in order to improve the performance [8], [9]. These designs require hundreds of milliseconds to perform path planning and consume a considerable amount of power, which is not feasible for battery-powered microrobots. In [10], a dedicated path planning processor that performs 2-D navigation based on the iterative-deepening A* (IDA*) algorithm [11] is proposed. IDA* is a grid-based algorithm that is used to identify the most-likely path on a map. The likelihood is determined by evaluating possible nodes based on the cost of traversing from the start point to the current node and the estimated cost to traverse from the current node to the endpoint. However, IDA* requires a massive consumption of memory in order to store the occupancy grid map and may need to repeatedly search for the same nodes. This causes an exponential growth in the complexity of both the memory storage and the search duration as the size of the map increases. For maps that have increased dimensions and resolution, IDA* usually fails to find a solution within an acceptable period. Consequently, the design presented in [10] only supports a 2-D map that has the grids of 100 × 100. If each grid is 10 × 10 cm^2, a 100 × 100-grid 2-D map is sufficient for a robot that is operating in a room. However, for large workspaces, such as a factory that is more than 100 m in length and width, the same 100 × 100-grid map contains 1 × 1 m^2 grids, which is insufficient for applications that demand high-resolution requirements.

In contrast, sampling-based algorithms [12]–[14], such as the rapidly exploring random tree (RRT) algorithm [15], allow for a more efficient approach to path planning for larger maps. The RRT algorithm constructs a tree structure that incrementally explores the surrounding space and eventually generates a path between the start point and the endpoint. In each iteration, an attempt is made to expand the tree by creating a new branch in randomly sampled directions. Over the years, several variants of the RRT algorithm have been developed for specific needs [16]–[19]. However, the hardware complexity

Manuscript received May 12, 2020; revised September 4, 2020 and November 4, 2020; accepted November 7, 2020. Date of publication November 24, 2020; date of current version December 24, 2020. This article was approved by Guest Editor Dejan Markovic. This work was supported in part by the Ministry of Science and Technology, Taiwan, under Grant MOST 108-2218-E-002-018 and in part by the Intelligent and Sustainable Medical Electronics Research Fund of National Taiwan University. *(Corresponding author: Chia-Hsiang Yang.)*

Chieh Chung was with the Graduate Institute of Electronics Engineering, National Taiwan University, Taipei 10617, Taiwan. He is now with MediaTek Inc., Hsinchu 30078, Taiwan.

Chia-Hsiang Yang is with the Department of Electrical Engineering, National Taiwan University, Taipei 10617, Taiwan, and also with the Graduate Institute of Electronics Engineering, National Taiwan University, Taipei 10617, Taiwan (e-mail: chyee@ntu.edu.tw).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSSC.2020.3037138.

Digital Object Identifier 10.1109/JSSC.2020.3037138

0018-9200 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

required to be implemented for these RRT algorithms is still excessively high. Only certain parts of a variant of the RRT algorithm, the RRT* algorithm [20], have been realized on field-programmable gate array (FPGA) [21]. The high amount of memory required to store the information for the tree nodes through direct hardware mapping also limits any capability.

This work presents an energy-efficient RRT-based path-planning processor that can be used for the autonomous navigation of microrobots. The chip is implemented using a 40-nm CMOS technology process and dissipates 1.5 μJ/task at a clock frequency of 200 MHz from a supply voltage of 0.9 V. The processor supports real-time path planning for both 2-D and 3-D maps while providing an improved energy efficiency that is greater than the current state-of-the-art designs. The key features of this work are summarized as follows.

1) A hardware-efficient RRT-based algorithm is proposed that supports efficient path planning for larger maps. For a 2-D map that contains 100 $\times$ 100 grids, the latency can be reduced by 99.9% compared with that achieved by implementing IDA*.
2) Hardware complexity is greatly reduced by employing dual-tree planning, branch extension, and parallel expansion. Computational complexity and the memory size required when building the exploration tree is reduced by 99.9% and 97.1%, respectively.
3) A prune-and-reuse replanning strategy is proposed in order to enable fast path replanning for dynamic environments.
4) An array of processing engines (PEs) that process the tree expansion in parallel is proposed, reducing the overall exploration time by 80.5%.
5) This work achieves an improvement that is three orders of magnitude greater in both latency and energy dissipation when compared to prior work.

The remainder of this article is organized as follows. In Section II, the planning algorithms are reviewed. The proposed techniques for the hardware-friendly RRT algorithm are described in Section III. An efficient re-planning strategy for dynamic environments is discussed in Section IV, whereas Section V presents the system architecture for the proposed path-planning processor. Chip implementation and an evaluation of the performance are introduced in Section VI. Finally, Section VII concludes this article.

II. Path -Planning Algorithms

Path planning [22] is used to identify a valid path that connects a start point and an endpoint while avoiding collisions with any obstacles that may be present. There are two major types of path-planning algorithms: grid-based and sampling-based. The grid-based algorithm divides the map into a number of grids and then converts the map into a graph. Conventional search algorithms are then applied to the graph to identify a valid path. Of the grid-based algorithms, IDA* [11] utilizes a best-first search strategy by estimating the distance from the current point to an endpoint. The search process terminates when the endpoint is reached. The determined path is the one that connects the start point and the endpoint at the minimum cost. However, the number of grids multiplies exponentially with the size of the map, and the IDA* may repeatedly explore the same nodes. Thus, as the computational complexity intensifies, the IDA* algorithm becomes increasingly infeasible for high-dimensional maps.

The sampling-based algorithm explores the map through random sampling. In the worst case scenario, the complexity is proportional to the total number of nodes, which is $\mathcal{O}(N^d)$, where N and d are the resolution and the dimensions of the map, respectively. RRT [15] is a sampling-based algorithm that incrementally constructs a tree structure, termed the exploration tree, to explore the space while simultaneously searching for the path. The root of the exploration tree is initially set at the start point. During each iteration, a node near the leaves of the current tree is randomly sampled in order to expand the exploration tree, and the closest node on the tree is then identified and denoted as its nearest neighbor. Subsequently, the location of a new node in the direction from the nearest neighbor toward the random node is identified. As long as this produces no collisions with any obstacles, the new node is added to the exploration tree. As the exploration tree expands, it will eventually connect to the endpoint. Thus, the RRT algorithm is able to determine a valid path that has a lower computational complexity. Fig. 1(a) shows a conceptual view of the RRT algorithm, and Fig. 1(b) shows the latencies for both the IDA* and RRT algorithms. As can be seen, the latency for the RRT algorithm is shorter, especially if the size of the map increases. For a 2-D map that contains 64 $\times$ 64 grids, the latency for the RRT algorithm can be reduced by 99.9% when compared to IDA*, with an increment of between 5% and 10% across the overall length of the path.

III. Computational Complexity Reduction

Unlike the IDA* algorithm, the computational complexity for RRT does not increase exponentially as the size of the map grows. However, the number of possible directions for expansion still expands for maps that have greater dimensions, making it difficult to identify a path that will terminate at the endpoint within a short timeframe. In addition, storing the data for the entire exploration tree requires a large amount of memory. To address these issues, three techniques are adopted so as to reduce the complexity of the RRT algorithm.

A. Dual-Tree RRT

In this work, path planning is further enhanced using the dual-tree planning strategy [23]–[25], where two sub-trees are simultaneously constructed. Fig. 2(a) shows the dual-tree algorithm, which includes two sub-trees rooted at both the start point and the endpoint. The expansion of the two sub-trees is guided in an interactive manner. Initially, one sub-tree is assigned as the master tree and the other is defined as a slave. During each iteration, the master tree is expanded using the same random sampling method as used for the single-tree RRT algorithm. The slave tree, however, expands toward the master tree by selecting nodes from the master tree

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

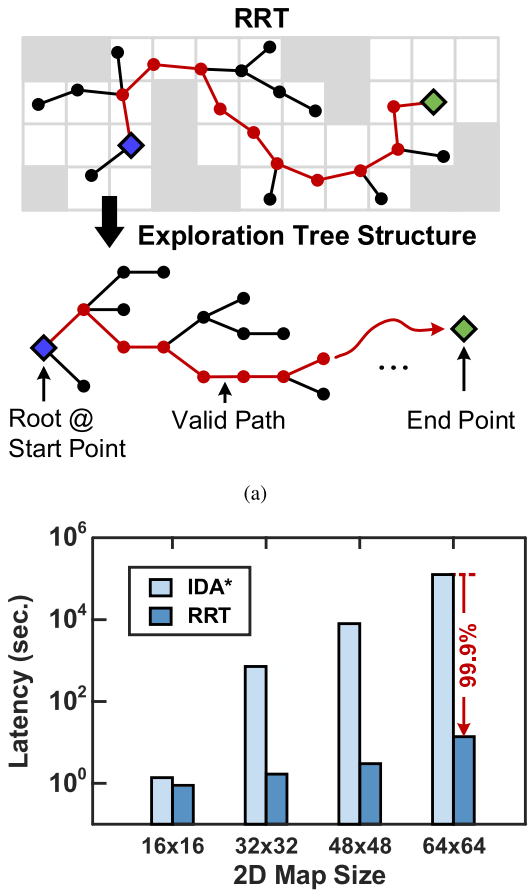


Fig. 1. (a) Exploration tree of RRT. The blue and green diamonds denote the start point and endpoint, respectively. The gray areas represent obstacles. (b) Latencies of IDA* and RRT on 2-D maps.

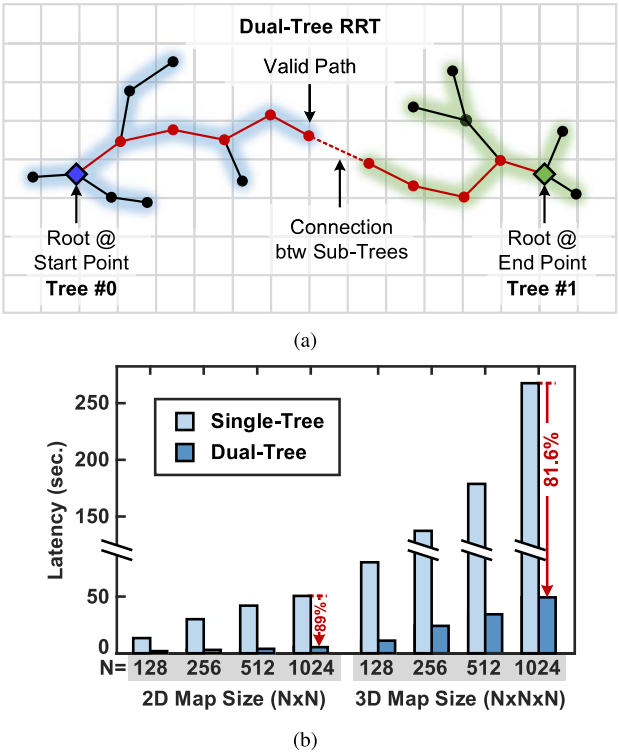


Fig. 2. (a) Dual-tree planning strategy, in which two sub-trees grow from the start point and endpoint. A valid path is found when the two sub-trees are connected. (b) Latencies of the single-tree and dual-tree methods.

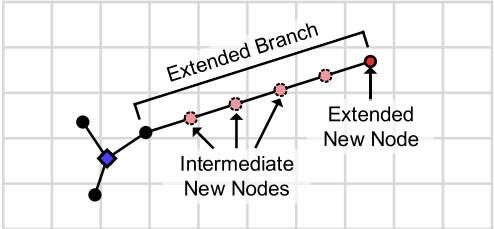


Fig. 3. Branch extension.

as expansion targets. The roles of the master and the slave trees exchange after a number of iterations so as to balance the construction of the dual-tree. In this manner, each sub-tree explores its surrounding space and eventually connects to each other. The two sub-trees simultaneously explore the map from both the start point and the endpoint, thereby accelerating the path-finding task. Fig. 2(b) shows the latencies for both the single-tree and the dual-tree methods. Experimental results show that, for both 2-D and 3-D maps that have various resolutions, the dual-tree RRT method is able to reduce the latency by over 80% compared with its single-tree counterpart.

B. Branch Extension

Conventionally, each new node is generated one step away from its nearest neighbor, given a random node during each iteration. Fig. 3 shows the construction of the RRT algorithm that includes branch extension. With branch extension, new nodes along the direction of the extension are generated and connected as long as no obstacles are encountered, once the first node is added to the exploration tree. When applied to the dual-tree RRT algorithm, branch extension allows two sub-trees to approach each other faster. Since each intermediate node is the nearest neighbor, the time required to search for the nearest neighbor is also reduced. Consequently, a longer branch can be constructed in a shorter period, and the extended branch is able to progress deeper into the unexplored spaces. The latency for space exploration can be therefore reduced. Moreover, the information for an extended branch can be efficiently represented by the two end nodes, without the need to store the intermediate data.

C. Parallel Expansion

For the conventional RRT algorithm, a random node is sampled, and only one new node is generated during each iteration. Thus, any newly connected branch only explores the map in a single direction. In contrast, an RRT algorithm that includes parallel expansion [31], [32] randomly samples multiple nodes during a single iteration. The nearest neighbor corresponding to each random node is then identified and used to generate a new node. Fig. 4 shows an example that illustrates parallel expansion, in which three new branches

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

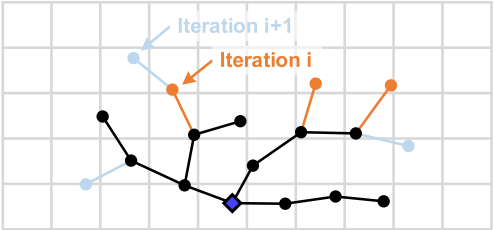


Fig. 4. Parallel expansion along multiple directions. Here, the number of directions is set to 3.

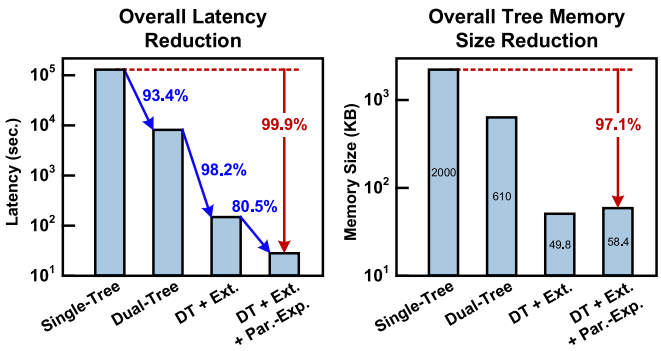


Fig. 5. Reductions in computational complexity and memory size of the exploration tree after applying dual-tree strategy (DT), branch extension (Ext.), and parallel expansion (Par.-Exp.).

expanding in different directions are generated during each iteration. The number of iterations required to explore the map is therefore reduced. The processing for multiple directions can be performed in parallel, thereby reducing the overall latency. The optimal hardware implementation for parallel expansion will be elaborated on in Section V.

D. Summary of Complexity and Memory Reduction

In this work, three planning strategies for reducing the computational complexity of the RRT algorithm are applied. Two sub-trees are created so as to accelerate the path-finding task, and multiple directions are also explored for each sub-tree during each iteration. The branch is extended whenever a new node is added to a sub-tree, as long as no obstacles are encountered. For a 3-D map that has a resolution of 32k grids, the experimental results show that the dual-tree strategy first reduces the latency by 93.4%, and then, branch extension further reduces the latency by 98.2%. Using a 32-way parallel expansion of the sub-trees as an example, an additional latency of 80.5% can be reduced, leading to an overall reduction in latency of 99.9% compared with that of the single-tree RRT algorithm. Although applying the dual-tree strategy increases the amount of computation needed in each iteration, the number of iterations required can also be significantly reduced due to the faster convergence speed achieved by the dual-tree strategy. It is noted the dual-tree RRT technique does not increase the required amount of energy, and the energy dissipation per task is actually reduced by 64% based on the experimental results. Overall, the proposed algorithms are able

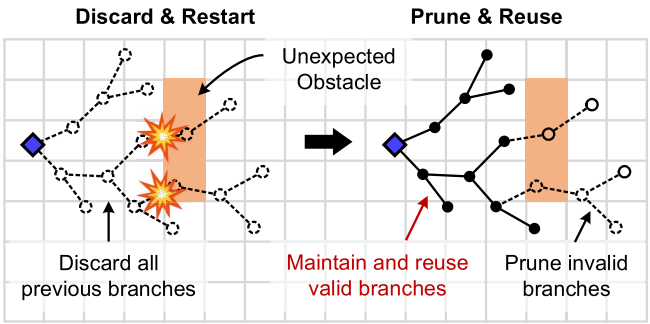


Fig. 6. Prune-and-reuse re-planning strategy. Invalid branches are removed, whereas the valid ones are maintained for finding a detour.

to reduce the amount of computation and energy dissipation by 91%.

The adopted strategies can also help to reduce the memory size for storing the exploration tree. Initially, the single-tree RRT approach requires about 2 MB of memory space to store the exploration tree. Since the two sub-trees can grow toward each other, fewer branches need to be constructed. The intermediate nodes that are needed to generate the branch extension can also be efficiently recorded. After applying the techniques, the size of the memory is reduced by 97.1%, and only 60 kB of memory space is needed. Since the amount of memory access is proportional to the latency and the memory required to store the tree structure, the dual-tree strategy also reduces the amount of memory access by 83%. In total, about 99% of the memory access is eliminated using the adopted techniques. In addition, from the experimental results, the accuracy in the overall path length is not affected, meaning that the final design produces the same range of errors after applying the optimizations. Fig. 5 summarizes the reductions in computational complexity and size of the memory required when building the exploration tree. The improvements in latency are average improvements on a set of 3-D test maps that have a resolution of 32 768 grids. Since there are no benchmarks in this field, a portion of the test maps is generated from a collection of the test cases demonstrated in related works [10], [23], [29], and others are generated by randomly placing obstacles of various sizes on the map.

IV. Re-Planning in Dynamic Environments

The RRT algorithm was originally designed as an approach to identifying a path in a static environment. However, autonomous microrobots often operate in an environment where obstacles unexpectedly appear. Therefore, these robots require an ability to quickly adapt to such a dynamic environment. Fig. 6 shows a discard and restart method that discards previous valid branches and starts again to identify an alternative route. However, the computational complexity for this brute-force-search method is high, introducing an extended processing latency.

Many studies have focused on efficient re-planning for sampling-based planning algorithms [26]. The underlying idea is to reuse the valid segments of previous branches in order to reconstruct a valid path. For instance, the execution

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

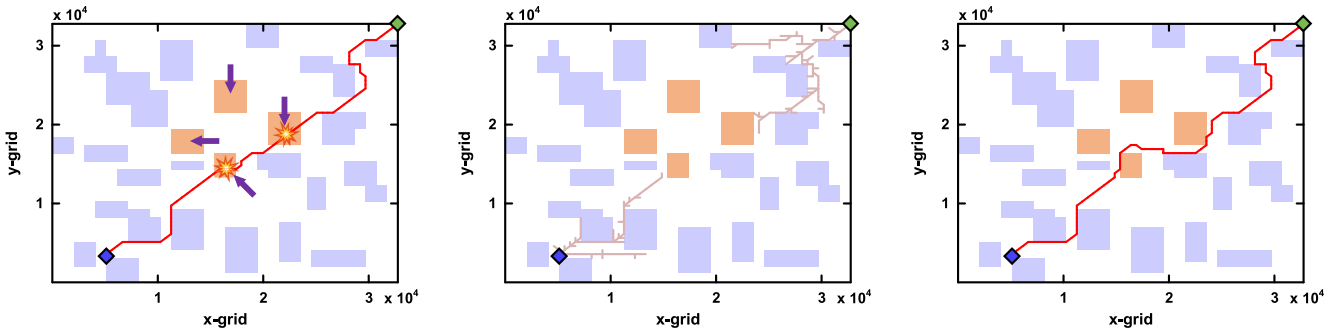


Fig. 7. Re-planning for RRT in a dynamic environment. (a) Original path with blocked branches. (b) Sub-trees with remaining valid branches. (c) Re-planned path.

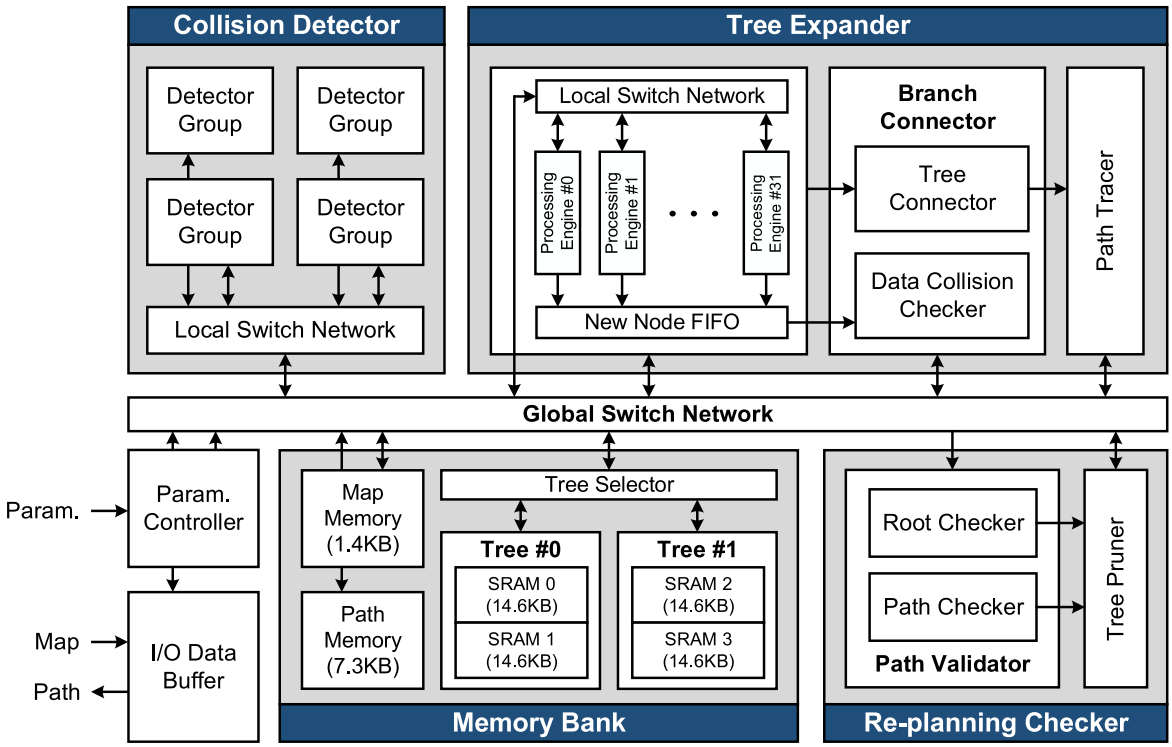


Fig. 8. System architecture for the proposed energy-efficient path planning processor.

extended RRT (ERRT) [27] method stores waypoints from early branches and re-plans the route based on previously visited nodes. The dynamic RRT (DRRT) [28] approach deletes any branches that encounter obstacles, and new branches are developed around the obstacles. Similarly, the multipartite RRT (MP-RRT) [29] and lazy reconfiguration forest (LRF) [30] methods remove any invalid branches and attempt to reconnect the remaining branches.

In this work, a prune-and-reuse strategy is adopted as an approach to path re-planning. As shown in Fig. 6, rather than entirely discarding the current tree, the prune-and-reuse strategy attempts to reconstruct the tree based on an updated map, and only those branches that are blocked by an obstacle are removed. The remaining valid branches are retained and used to identify a detour. Fig. 7 shows an example of re-planning for the RRT method in a dynamic environment. Initially, a path is identified by the RRT algorithm. As the robot begins moving along the path, a number of obstacles also move and block the path, as shown in Fig. 7(a). In this situation, the branches on the exploration tree are examined. Any blocked branches are removed, while the remainder are retained, as shown in Fig. 7(b). By reusing the valid branches, a detour that bypasses the obstacles can be identified, as shown in Fig. 7(c).

V. System Architecture

Fig. 8 shows the system architecture for the proposed path planning processor. The processor consists of a tree expander, a collision detector, a re-planning checker, a memory bank, and a global switch network. The tree expander performs the path planning for the RRT algorithm based on parallel processing. The data collision checker examines the newly

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

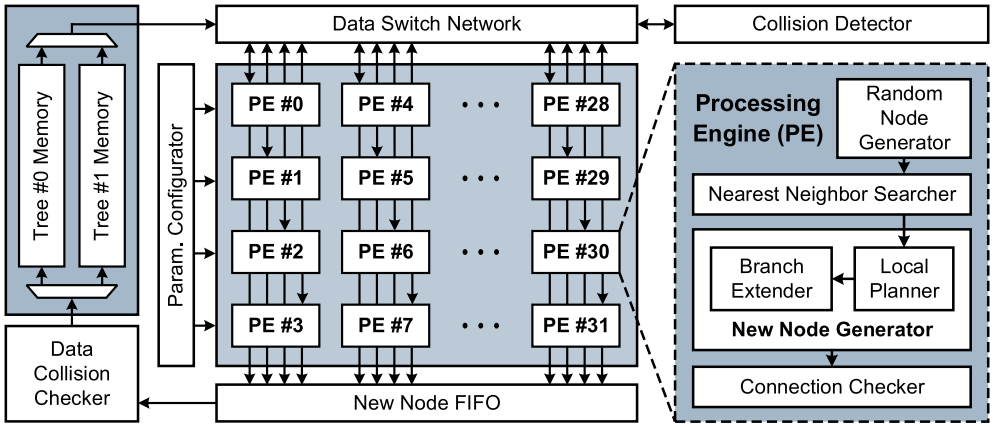


Fig. 9. Details of the array of PEs.

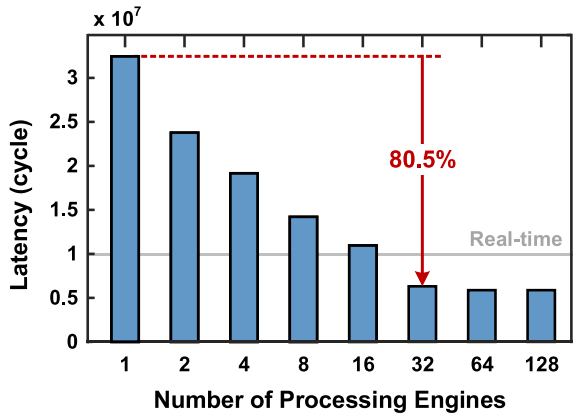


Fig. 10. Latency analysis for the optimal number of PEs. The real-time specification is set to 50 ms, based on the design in [10].

generated nodes and adds them to the exploration tree. The tree connector conditionally connects any sub-trees using a greedy approach in order to determine the final path. The function of the collision detector is to identify any obstacles and determine a method of avoiding them by checking the validity of the newly generated nodes. If an unexpected obstacle is detected, the re-planning checker first examines the validity of the original path, and the collision detector assists in checking the validity of each branch. If the original path is blocked, a tree pruner removes any invalid branches while retaining valid branches. Subsequently, newly added valid branches are constructed in order to identify any potential detours around the obstacles. The coordinates of the obstacles, the exploration trees, and the path points are then stored in the memory bank. The global switch network allows the data to be transmitted between each block via dedicated wires. For instance, the tree expander is able to fetch data from the tree memory and forward details of any nodes to be checked to the collision detector via specialized routes.

A. Tree Expander

The tree expander is the most computationally intensive aspect of the path planning process, performing the construction of the exploration tree. The tree expander receives data from the tree memory and communicates with the collision detector in order to generate any new nodes. After passing the data collision check, any generated new nodes are collected using a new node FIFO and added to the exploration tree. The tree expander is then shared so as to assist in the expansion of the two sub-trees, thereby reducing the area required by 50%.

The core of the tree expander is an array of PEs that simultaneously perform the tree expansion in multiple directions. The PEs are executed in parallel and collaborate in building a sub-tree. Fig. 9 shows the details of the array of PEs, and the optimal number of PEs is determined through latency analysis. As shown in Fig. 10, the latency decreases as the number of PEs increases but saturates once there are 32 PEs. This is because, with that many PEs, identical directions for expansion are often identified, so the latency cannot be further improved. Consequently, a design that contains 32 PEs is chosen so as to minimize the latency while requiring the smallest area. A PE is responsible for the growth of the tree in a single direction for parallel expansion. It is also designed to perform the entire RRT algorithm in order to provide the highest flexibility. With a maximum of 32 newly added nodes in a single iteration, up to 32 directions can be simultaneously explored in order to expand the tree, thereby reducing the overall latency by 80.5%.

B. Processing Engine

A PE is able to perform tree expansion for the RRT algorithm, including generating a random node, searching for the nearest neighbor, generating a new node, and extending the newly connected branch. A connection checker is also included so as to detect whether the newly generated node is within a sufficient distance in order to connect with the other sub-tree. The linear feedback shift registers are used to generate a random number. From the experimental results, the range of difference in the overall path length between the pseudorandom number generators and the true random numbers is about $\pm 2.5\%$. In addition, generating an in-processing element random number is necessary so as to perform re-planning in dynamic environments. This allows different random sequences to be generated and to construct

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

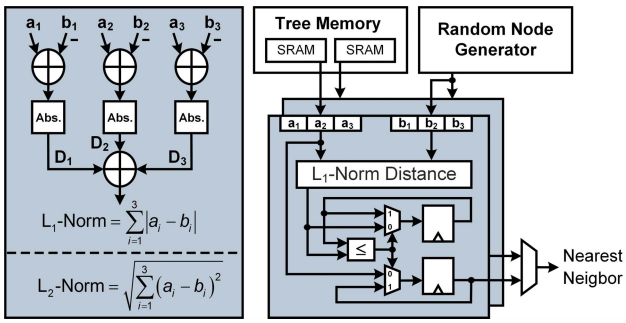


Fig. 11. Details of the nearest neighbor searcher.

new branches in a direction that is different from the original path. The PE array within the tree expander dominates the overall processor area. In particular, the nearest neighbor searcher and the local planner consumes up to three quarters of the area for the array. Consequently, optimization techniques are applied so as to reduce the complexity of the hardware.

1) Nearest Neighbor Searcher: Fig. 11 shows the nearest neighbor searcher, which computes the distances between a random node and the nodes on the tree in order to locate the nearest neighbor. In this work, L1-norm, rather than the conventional L2-norm, is used to compute the distance, reducing the complexity of the hardware by 57.5%. Adopting the L1-norm to reduce the hardware complexity does not compromise the performance thanks to the random nature of the RRT algorithm. From the experimental results, the range of errors is ±5% with respect to the path length. To further accelerate the operations required to access and process all the tree nodes, the memory block for each sub-tree is partitioned into two banks to allow for access in parallel, and the data are equally distributed in these banks. This allows the nearest neighbor searcher to simultaneously access and process two separate nodes, reducing the runtime for this stage by 50%.

2) Local Planner: A local planner is designed such that it generates a new node once the nearest tree node is identified. Conventionally, the location of a new node (c_1', c_2', c_3') is determined by moving the nearest neighbor along the path in the direction of vector $\vec{v}$, starting at the nearest neighbor and ending at the random node, based on a step size s, as given by

$$(c_1', c_2', c_3') = \frac{s}{|\vec{v}|} \cdot \vec{v} + (a_1, a_2, a_3) \tag{1}$$

$$\vec{v} = (b_1 - a_1, b_2 - a_2, b_3 - a_3) \tag{2}$$

$$|\vec{v}| = \sqrt{\sum_{i=1}^{3} (b_i - a_i)^2} \tag{3}$$

where (a_1, a_2, a_3) and (b_1, b_2, b_3) are the coordinates of the nearest neighbor and the random node, respectively. This step involves square root operation and division and needs to be frequently performed. In this work, a discretized selection method is proposed so as to simplify this step.

Fig. 12 shows the details of the selection method, where D_i represents the absolute distance between a_i and b_i for the ith dimension. The possible locations for the new node are

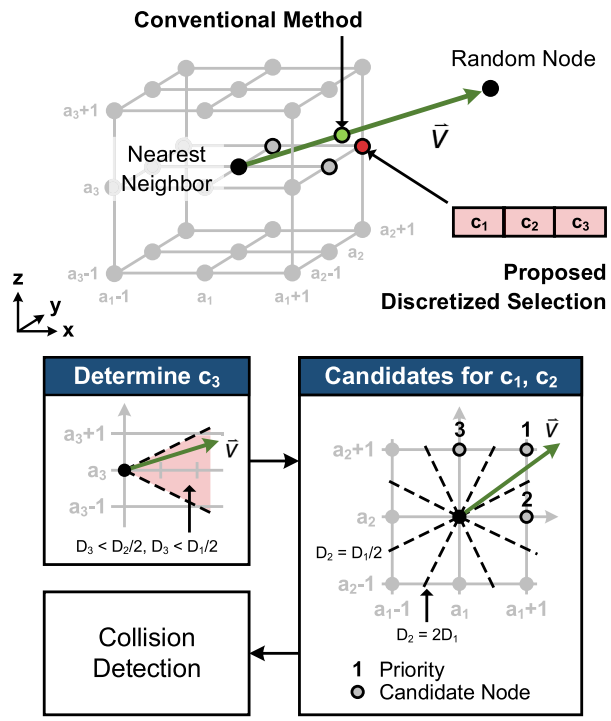


Fig. 12. Proposed discretized selection method for local planner.

limited to the integer coordinates around the nearest neighbor. Therefore, the search space only includes the gray nodes in the cube. The coordinates for the new node (c_1, c_2, c_3) can be estimated in an efficient manner: 1) c_3 is determined based on the direction of $\vec{v}$ along the z-axis, which is either $a_3 + 1$, a_3 or $a_3 - 1$; 2) c_1 and c_2 are determined based on the orientation of $\vec{v}$ when projected onto the xy-plane. Therefore, c_1 is either $a_1 + 1$, a_1, or $a_1 - 1$, and c_2 can be determined in a similar manner. The selected coordinates are then provided to the collision detector to determine whether any collisions exist. If no collisions with any obstacles are detected, the new node is connected to the nearest neighbor to form a new branch. To increase the possibility that a valid new node is generated, several candidate locations are provided. The proposed method reduces the search space and quickly identifies the point on the path that is closest in the direction of the random node. The complexity of the hardware for the local planner is reduced by 65.5% without any loss in performance.

C. Collision Detector

In this work, obstacles are formulated as rectangles on 2-D maps and as cubes on 3-D maps. A node is considered invalid if its coordinates lie in the space occupied by any obstacle. A collision detector is composed of four clusters, and each cluster includes 32 processing elements. Each processing element compares the location of a given node with that of an obstacle when performing a collision check. The map memory is partitioned into four sections in order to allow parallel data access to four clusters, achieving a 4× reduction in latency when considering collision detection. It is noted that the collision detector is shared by two operations: tree expansion

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

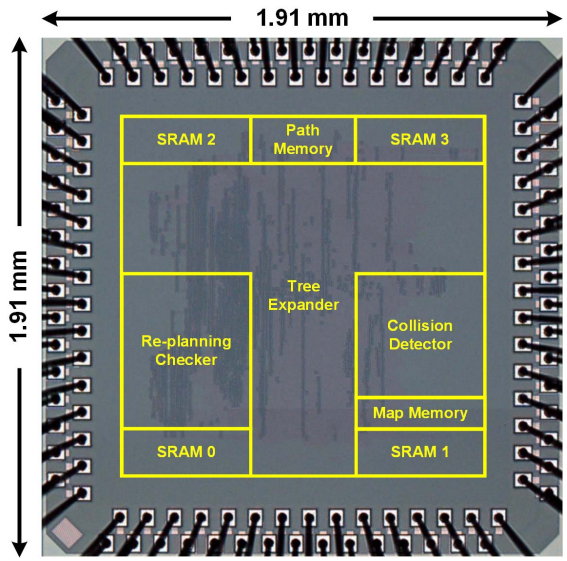


Fig. 13. Chip micrograph.

TABLE I
CHIP SUMMARY

Technology		40nm-HP 1P9M CMOS
Area (mm^2)	Die	1.91×1.91
	Core	1.53×1.53
Operation Voltage (V)		0.9
Max. Frequency (MHz)		200
On-Chip Memory (KB)		84
Map Size	2D	Up to 32768^2
	3D	Up to 32768^3
Processing Latency† (ms/task)	2D	0.015-to-0.125
	3D	0.04-to-7.5
Power (mW)	2D	102.6
	3D	109.9
Energy† (μJ/task)	2D	1.5-to-12.8
	3D	4.4-to-824.3

†Planning performance on maps with a resolution of 100-to-32768 in each dimension.

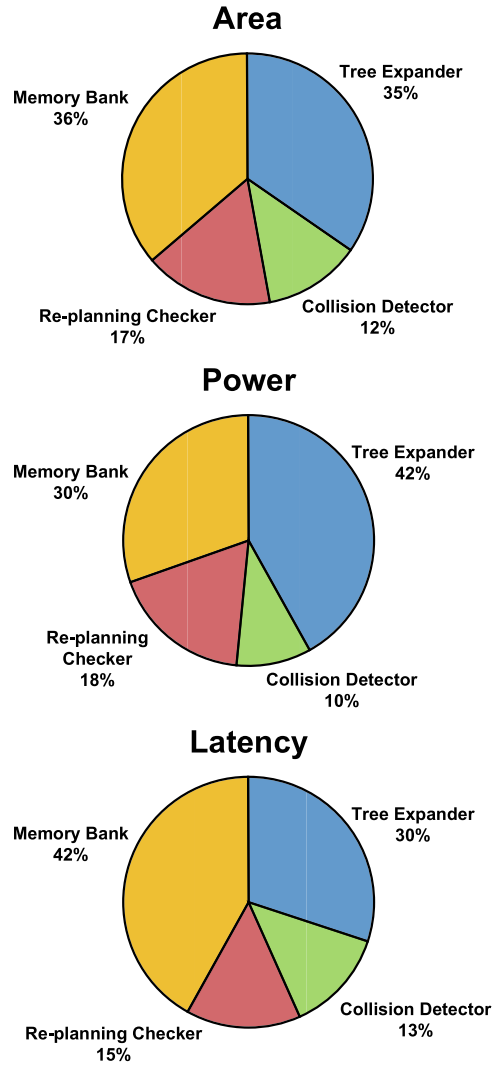


Fig. 14. Power, area, and latency breakdowns.

and tree pruning. For the tree expansion, every processing element in the same cluster performs a collision check on the new nodes. For the tree pruning, only half of the processing elements in the same cluster perform a collision check on the branches of each sub-tree. This mapping scheme achieves the maximum utilization of the processing elements and also reduces any latency when detecting unexpected obstacles in a dynamic environment.

D. Re-Planning Checker

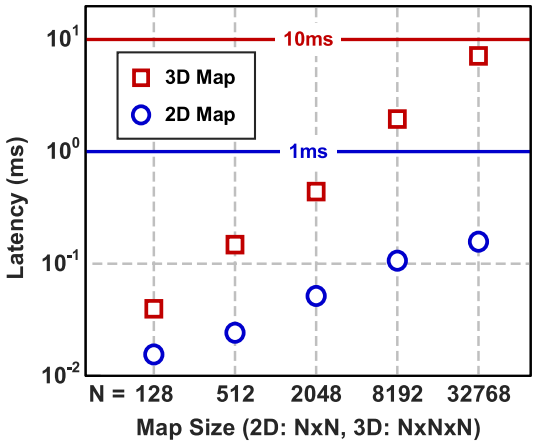
The re-planning checker is composed of a path validator and a tree pruner. The path validator is activated once the map is updated and checks the connections between each of the path nodes to verify whether or not the connections are valid. If the original path is blocked by an obstacle, tree reconstruction is performed by the tree pruner. Since the tree nodes are incrementally added, a parent node is always processed prior to its child nodes. If a parent node is valid, a collision check is performed on its child nodes and associated branches. Otherwise, collision checks for the succeeding child nodes can be terminated. The validity of a branch can be verified by checking the status of the parent node that is stored in a buffer, and the status of any child nodes is only stored in the tree memory if they remain valid. The two sub-trees are stored in separate SRAMs so that they can be simultaneously processed. After tree pruning is performed, only those tree nodes that remain valid are retained in the tree memory. The tree expander then continues to explore and add new nodes to the exploration tree in order to identify an alternate path.

VI. EXPERIMENTAL VERIFICATION

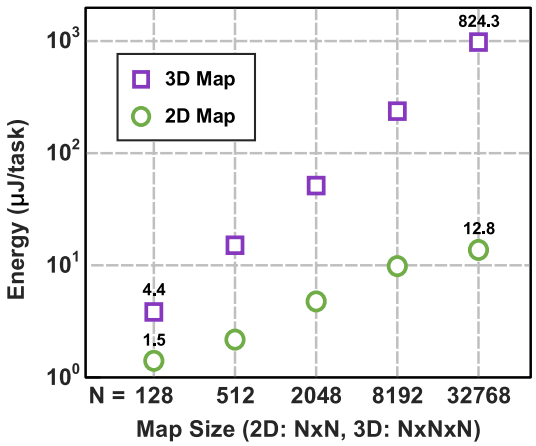
A. Chip Implementation

Fig. 13 shows the micrograph for the proposed path planning processor. The chip characteristics are summarized in Table I. Fabricated based on a 40-nm CMOS technology

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

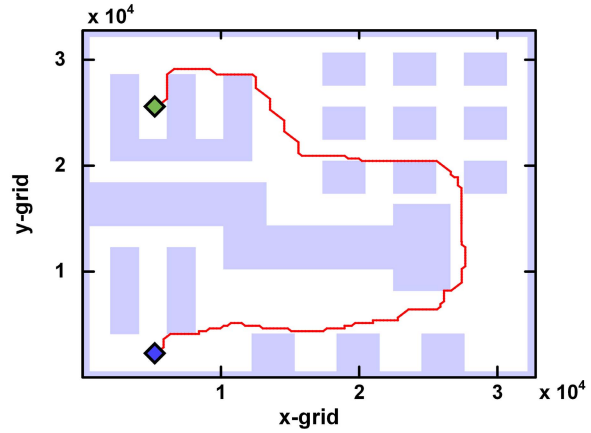


(a)

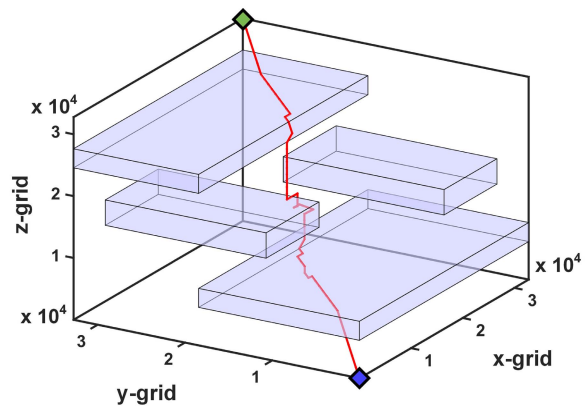


(b)

Fig. 15. (a) Planning latencies and (b) energy dissipation for 2-D and 3-D maps with various resolutions. It takes less than 1 and 10 ms to perform path planning tasks in the 2-D and 3-D maps respectively, and dissipates only 1.5 μJ per task for a 128 × 128 2-D map.



(a)



(b)

Fig. 16. Path planning examples demonstrating the effectiveness for (a) 2-D and (b) 3-D maps with a resolution of 32 768 in each dimension.

process, the chip integrates 2.02M logic gates in an area of 3.65 mm^2 [33]. The maximum clock frequency is 200 MHz from a 0.9-V supply voltage. The chip supports both 2-D and 3-D planning tasks, with a power dissipation of 103–110 mW. Fig. 14 shows the power, area, and latency breakdown for the processor. The power and latency breakdown is for a 3-D map with a resolution of 32 768 grids (in each dimension) in a dynamic environment. The total power is measured and the power breakdown is based on post-layout estimates. The average consumption required by the tree expander is 42% of the total power since it is the most computationally intensive unit within the processor. The tree expander consumes 35% of the total area and 30% of the total latency. In addition, the memory access time is 42% of the total latency since several steps in the RRT algorithm require fetching data stored in the memory bank, including for the tree nodes required in the nearest neighbor search and the position of the obstacles when performing collision detection. The power and latency breakdown greatly depends on a given task. For instance, the tree expander becomes even more dominant and the collision detector becomes even less significant for a smaller 2-D map or for a path with more turning points. Moreover, the latency and power taken by the re-planning checker depend on how dynamic the environment is.

B. Performance Evaluation

Fig. 15(a) shows the planning latency for 2-D and 3-D maps that have a width, depth, and height at various resolutions, i.e., 128, 512, 2048, 8192, and 32 768 grids, when operating at the maximum frequency. This work supports path planning for both 2-D and 3-D maps containing up to 32 768 grids in each dimension, with a latency of less than 1 and 10 ms, respectively. Fig. 15(b) shows the corresponding energy dissipation per task. The chip dissipates only 1.5 μJ per task for a 2-D map that has a grid of 128 × 128, which is a similar case tested in the prior art. Due to the high scalability, this work is able to support a 3-D map with up to 32 768 grids in each dimension, and the associated energy consumption is 824.3 μJ per task.

Fig. 16 shows the examples of the path planning for 2-D and 3-D maps that have a resolution of 32 768 grids in each dimension. The maps are designed to mimic a complex workspace. The processor is able to plan a valid path that connects the start point and endpoint in real time while successfully avoiding

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE COMPARISON

Design		ISSCC'16 [10]	ISSCC'19 [34]	This Work
Technology		65nm	22nm	40nm
Planning Algorithm		IDA*	-	RRT
Map Size	2D	Up to 100^2	-	**Up to 32768^2**
	3D	-	-	**Up to 32768^3**
Area (mm^2)	Die	4×4	-	1.91×1.91
	Core	3.53×3.53 (est.)	1.27×1.02 (est.)	**1.53×1.53**
On-Chip Memory (KB)		891	28	84
Operation Voltage (V)		1.2	1.0	0.9
Max. Frequency (MHz)		245	160	200
Processing Latency (ms/task)		22†	2	**0.015†**
Power Consumption (mW)		151	37.5	102.6
Energy (μJ/task)		3200†	24	**1.5†**

†Planning performance on 100×100 2D maps.

collisions with any obstacles. If the position of an obstacle changes within the environment, the map memory is updated accordingly. Following re-planning, an alternative path can be constructed.

C. Performance Comparison

Table II provides a performance comparison of the proposed work with the state-of-the-art path planning processors [10], [34]. The design presented in [10] is the first path-planning processor designed for indoor navigation, where the IDA* technique is implemented, and dissipates 151 mW at 245 MHz. However, it is possible that the algorithm may fail to identify a path within reasonable period, i.e., ≤50ms for a 2-D map that contains more than 100 grids in each dimension. In [34], a path planning accelerator is integrated into a robot SoC to perform search and rescue missions, consuming 37.5 mW at a clock frequency of 160 MHz. The SoC dissipates 24 μJ for a path-planning task. However, the details for this processor are not fully revealed and the size of the map is also not disclosed, making it difficult to conduct a fair comparison. This work presents an RRT-based path planning processor that can support both 2-D and 3-D planning tasks. The largest map that it can support is a 3-D map that contains 32 768 grids in each dimension. For a 2-D map that includes 100 × 100 grids, the proposed processor requires only 15 μs to perform a planning task, dissipating around 1.5 μJ/task, achieving improvements of 1467× and 2133× in processing latency and energy dissipation, respectively, based on a normalized area of 50.4% compared to [10]. Assuming that the design in [34] is based on a 2-D map that contains 100 × 100 grids or is of a similar size, this work still achieves an improvement of 16× in terms of energy dissipation. The latency of 15 μs indicated in Table II is used to demonstrate the superiority of the proposed processor. When a latency of 50 ms is sufficient [10], the clock frequency and the associated supply voltage required by this design can be reduced in order to further minimize the energy consumption.

VII. CONCLUSION

This article presents an energy-efficient path planning processor for 2-D/3-D autonomous navigation based on optimization of the algorithm architecture, where the sampling-based RRT algorithm is implemented in the processor. However, a small distance error is traded in order to achieve a great reduction in complexity when compared with grid-based algorithms, such as IDA*. The performance of the processor is enhanced by utilizing a dual-tree planning strategy. A combination of branch extension and parallel expansion further reduces both the computational complexity and the memory requirements, enabling efficient hardware implementation. The prune-and-reuse re-planning strategy allows the processor to quickly adapt to a dynamically changing environment and includes an array of PEs that are designed to accelerate the tree exploration process based on parallel processing. The optimal number of PEs is determined through latency analysis so as to approach the minimal latency based on the smallest hardware cost. Low-complexity operations that search for the nearest neighbor and select new nodes are adopted in order to minimize the hardware complexity. The proposed path-planning processor achieves improvements in processing latency and energy dissipation that are orders-of-magnitude greater compared with prior designs. The processor is also scalable and can support 3-D maps that include higher resolutions, providing a promising solution for real-time autonomous navigation by microrobots.

ACKNOWLEDGMENT

The authors thank the Taiwan Semiconductor Research Institute (TSRI), Taiwan, for technical support on chip design and fabrication.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] R. Bloss, "Mobile hospital robots cure numerous logistic needs," *Ind. Robot, Int. J.*, vol. 38, no. 6, pp. 567–571, Oct. 2011.

[2] N. Mathew, S. L. Smith, and S. L. Waslander, "Planning paths for package delivery in heterogeneous multirobot teams," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 4, pp. 1298–1308, Oct. 2015.

[3] N. Agmon, G. A. Kaminka, and S. Kraus, "Multi-robot adversarial patrolling: Facing a full-knowledge opponent," *J. Artif. Intell. Res.*, vol. 42, pp. 887–916, Dec. 2011.

[4] S. Witwicki *et al.*, "Autonomous surveillance robots: A decision-making framework for networked muiltiagent systems," *IEEE Robot. Autom. Mag.*, vol. 24, no. 3, pp. 52–64, Sep. 2017.

[5] S. Thrun *et al.*, "Autonomous exploration and mapping of abandoned mines," *IEEE Robot. Autom. Mag.*, vol. 11, no. 4, pp. 79–91, Dec. 2004.

[6] M. Bajracharya, M. W. Maimone, and D. Helmick, "Autonomy for mars rovers: Past, present, and future," *Computer*, vol. 41, no. 12, pp. 44–50, Dec. 2008.

[7] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*. Cambridge, MA, USA: MIT Press, 2011.

[8] J. Bialkowski, S. Karaman, and E. Frazzoli, "Massively parallelizing the RRT and the RRT," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Sep. 2011, pp. 3513–3518.

[9] C. Park, J. Pan, and D. Manocha, "Real-time optimization-based planning in dynamic environments using GPUs," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2013, pp. 4090–4097.

[10] Y. Kim, D. Shin, J. Lee, Y. Lee, and H.-J. Yoo, "A 0.55 V 1.1 mW artificial-intelligence processor with PVT compensation for micro robots," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2016, pp. 258–259.

[11] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," *Artif. Intell.*, vol. 27, no. 1, pp. 97–109, Sep. 1985.

[12] S. R. Lindemann and S. M. LaValle, "Current issues in sampling-based motion planning," in *Proc. 11th Int. Symp. Robot. Res.*, Oct. 2003, pp. 36–54.

[13] K. I. Tsianos, I. A. Sucan, and L. E. Kavraki, "Sampling-based robot motion planning: Towards realistic applications," *Comput. Sci. Rev.*, vol. 1, no. 1, pp. 2–11, Aug. 2007.

[14] I. A. Sucan and L. E. Kavraki, "On the implementation of single-query sampling-based motion planners," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2010, pp. 2005–2011.

[15] S. M. LaValle and J. J. Kuffner, "Randomized kinodynamic planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 1999, pp. 473–479.

[16] L. G. D. O. Veras, F. L. L. Medeiros, and L. N. F. Guimaraes, "Systematic literature review of sampling process in rapidly-exploring random trees," *IEEE Access*, vol. 7, pp. 50933–50953, 2019.

[17] L. Ma, J. Xue, K. Kawabata, J. Zhu, C. Ma, and N. Zheng, "Efficient sampling-based motion planning for on-road autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 4, pp. 1961–1976, Aug. 2015.

[18] J.-H. Park, W. J. Park, C. Lee, M. Kim, S. Kim, and H. J. Kim, "Endoscopic camera manipulation planning of a surgical robot using rapidly-exploring random tree algorithm," in *Proc. 15th Int. Conf. Control, Autom. Syst. (ICCAS)*, Oct. 2015, pp. 1516–1519.

[19] N. Seegmiller, J. Gassaway, E. Johnson, and J. Towler, "The maverick planner: An efficient hierarchical planner for autonomous vehicles in unstructured environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2017, pp. 2018–2023.

[20] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, Jun. 2011.

[21] S. Xiao, N. Bergmann, and A. Postula, "Parallel RRT architecture design for motion planning," in *Proc. 27th Int. Conf. Field Program. Logic Appl.*, Sep. 2017, pp. 1–4.

[22] S. M. Lavalle, *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ., 2006.

[23] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *Proc. ICRA Millennium Conf. IEEE Int. Conf. Robot. Automat. Symp.*, Apr. 2000, pp. 995–1001.

[24] S. M. LaValle and J. J. Kuffner, "Randomized kinodynamic planning," *Int. J. Robot. Res.*, vol. 20, no. 5, pp. 378–400, May 2001.

[25] D. Hsu, J.-C. Latombe, and R. Motwani, "Path planning in expansive configuration spaces," *Int. J. Comput. Geometry Appl.*, vol. 9, nos. 4–5, pp. 495–512, Aug. 1999.

[26] A. Short, Z. Pan, N. Larkin, and S. van Duin, "Recent progress on sampling based dynamic motion planning algorithms," in *Proc. IEEE Int. Conf. Adv. Intell. Mechatronics (AIM)*, Jul. 2016, pp. 1305–1311.

[27] J. Bruce and M. Veloso, "Real-time randomized path planning for robot navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, vol. 3, Oct. 2002, pp. 2383–2388.

[28] D. Ferguson, N. Kalra, and A. Stentz, "Replanning with RRTs," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2006, pp. 1243–1248.

[29] M. Zucker, J. Kuffner, and M. Branicky, "Multipartite RRTs for rapid replanning in dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom.*, Apr. 2007, pp. 1603–1609.

[30] R. Gayle, K. R. Klingler, and P. G. Xavier, "Lazy reconfiguration forest (LRF)—An approach for motion planning with multiple tasks in dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom.*, Apr. 2007, pp. 1316–1323.

[31] S. A. Jacobs, N. Stradford, C. Rodriguez, S. Thomas, and N. M. Amato, "A scalable distributed RRT for motion planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2013, pp. 5088–5095.

[32] D. Devaurs, T. Simeon, and J. Cortes, "Parallelizing RRT on distributed-memory architectures," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2011, pp. 2261–2266.

[33] C. Chung and C.-H. Yang, "A 1.5μJ/Task path-planning processor for 2D/3D autonomous navigation of micro robots," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 324–325.

[34] V. Honkote *et al.*, "2.4 a distributed autonomous and collaborative multi-robot system featuring a low-power robot SoC in 22 nm CMOS for integrated battery-powered minibots," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 48–49.

Chieh Chung (Graduate Student Member, IEEE) received the B.S. degree in electrical engineering and the M.S. degree in electronics engineering from National Taiwan University, Taipei, Taiwan, in 2017 and 2019, respectively.

His research interests include algorithms development and VLSI design for artificial intelligence (AI), robotics, and machine learning applications.

Chia-Hsiang Yang (Senior Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from National Taiwan University, Taipei, Taiwan, in 2002 and 2004, respectively, and the Ph.D. degree from the Department of Electrical Engineering, University of California at Los Angeles, Los Angeles, CA, USA, in 2010.

He then joined the Faculty of the Electronics Engineering Department, National Chiao Tung University, Hsinchu, Taiwan. In 2015, he moved to National Taiwan University, where he is currently a Full Professor. His research interests include energy-efficient integrated circuits and architectures for biomedical and communication signal processing.

Dr. Yang was a winner of the DAC/ISSCC Student Design Contest in 2010. He received the 2010–2011 Distinguished Ph.D. Dissertation in Circuits and Embedded Systems Award from the Department of Electrical Engineering, University of California at Los Angeles. In 2013, he was a co-recipient of the ISSCC Distinguished-Technical-Paper Award and the Demonstration Session Certification of Recognition. In 2018, he was a recipient of the Ta-You Wu Memorial Award from the Ministry of Science and Technology (MOST), Taiwan. He is also the advisor for several student awards, including the 2017 ISSCC Silkroad Award.

Authorized licensed use limited to: National Yang Ming Chiao Tung University. Downloaded on February 11,2026 at 23:13:33 UTC from IEEE Xplore. Restrictions apply.